

Now, let us see how we can publish an image to the Docker Hub Registry. For this, run the following commands:

```
$docker commit <container_name> <docker_
hub_repository>/<new_image_name>:<image_
tag>
$docker login
$docker push <image-name>:<tag>
```

Note that these commands can be used to publish the image to any Docker Registry.

Figure 2 shows how an image can be published to the Docker Hub Registry.

Let us explore this terminal by trying some other commands. Figures 3 and 4 show some basic Linux commands that do not run on Docker Quickstart Terminal and those that run on the terminal, respectively.

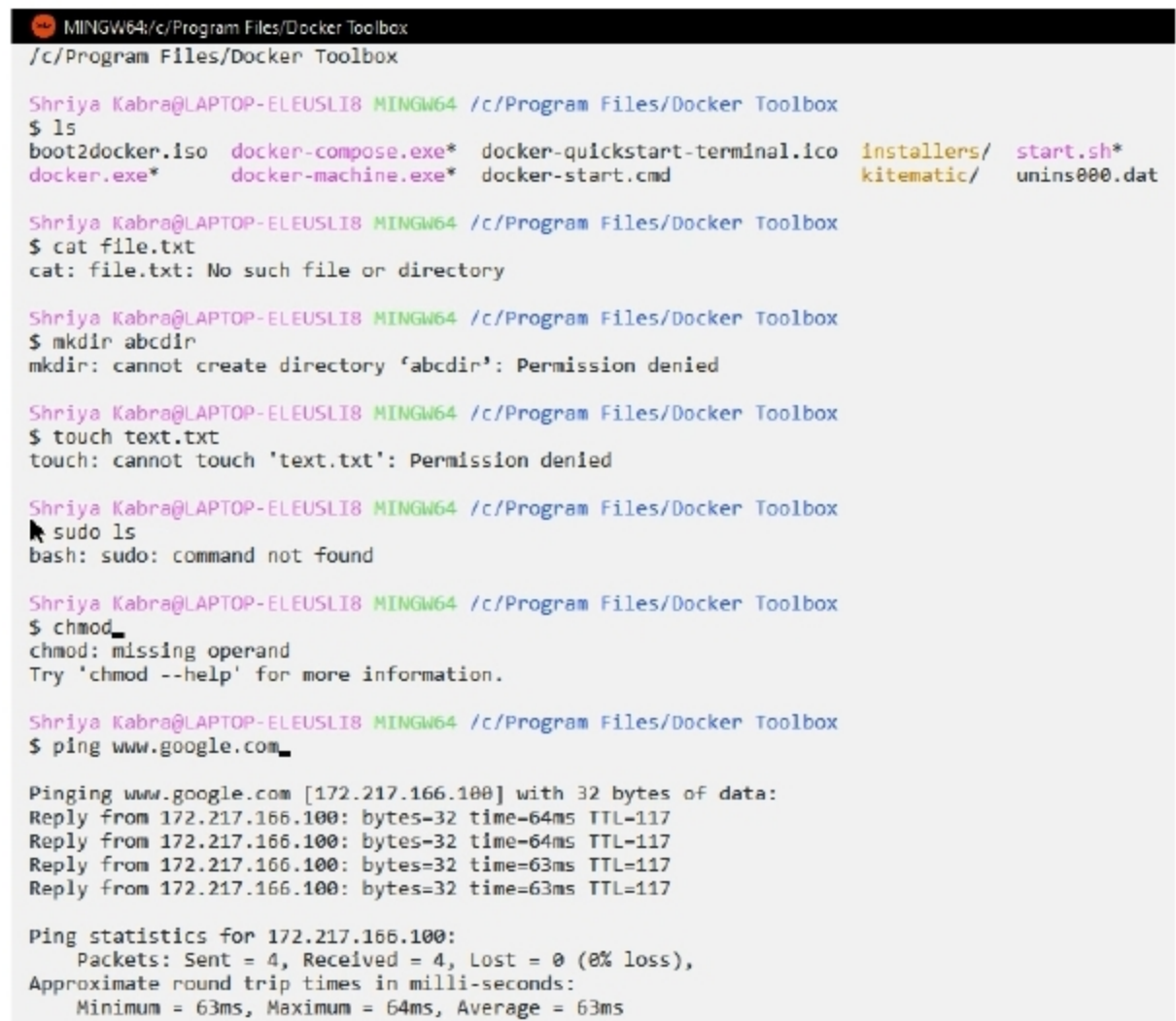
Kitematic (Alpha)

This is for software engineers who are not at ease with the CLI and Docker commands. Kitematic offers a GUI for working with Docker images and containers. Most of the basic functions can be executed with it. For using it, you must create a virtual machine when asked, and login with your Docker Hub ID.

In the left panel of Kitematic GUI, you will find the list of all stopped and running containers. The main panel lists all the recommended images from Docker Hub and the images in your own repository too. We can search for images and create new containers by clicking on *Create*. We can also specify port numbers, version, and tag of the image that we wish to pull.

On selecting a particular container in the left panel, a new view shows up. As we have run a Jenkins image on port number 8080, we can see the container logs in the main panel and *Web Preview* on the top right panel. Volumes are shown in the lower right panel. Output of container logs is shown in Figure 5.

We can also view and update container properties from the *Settings* panel. *Settings* shows all the container properties like container name,



```

MINGW64/c:/Program Files/Docker Toolbox
/c:/Program Files/Docker Toolbox

Shriya Kabra@LAPTOP-ELEUSLI8 MINGW64 /c:/Program Files/Docker Toolbox
$ ls
boot2docker.iso  docker-compose.exe*  docker-quickstart-terminal.ico  installers/  start.sh*
docker.exe*      docker-machine.exe*  docker-start.cmd                kitematic/  unins000.dat

Shriya Kabra@LAPTOP-ELEUSLI8 MINGW64 /c:/Program Files/Docker Toolbox
$ cat file.txt
cat: file.txt: No such file or directory

Shriya Kabra@LAPTOP-ELEUSLI8 MINGW64 /c:/Program Files/Docker Toolbox
$ mkdir abcdir
mkdir: cannot create directory 'abcdir': Permission denied

Shriya Kabra@LAPTOP-ELEUSLI8 MINGW64 /c:/Program Files/Docker Toolbox
$ touch text.txt
touch: cannot touch 'text.txt': Permission denied

Shriya Kabra@LAPTOP-ELEUSLI8 MINGW64 /c:/Program Files/Docker Toolbox
$ sudo ls
bash: sudo: command not found

Shriya Kabra@LAPTOP-ELEUSLI8 MINGW64 /c:/Program Files/Docker Toolbox
$ chmod_
chmod: missing operand
Try 'chmod --help' for more information.

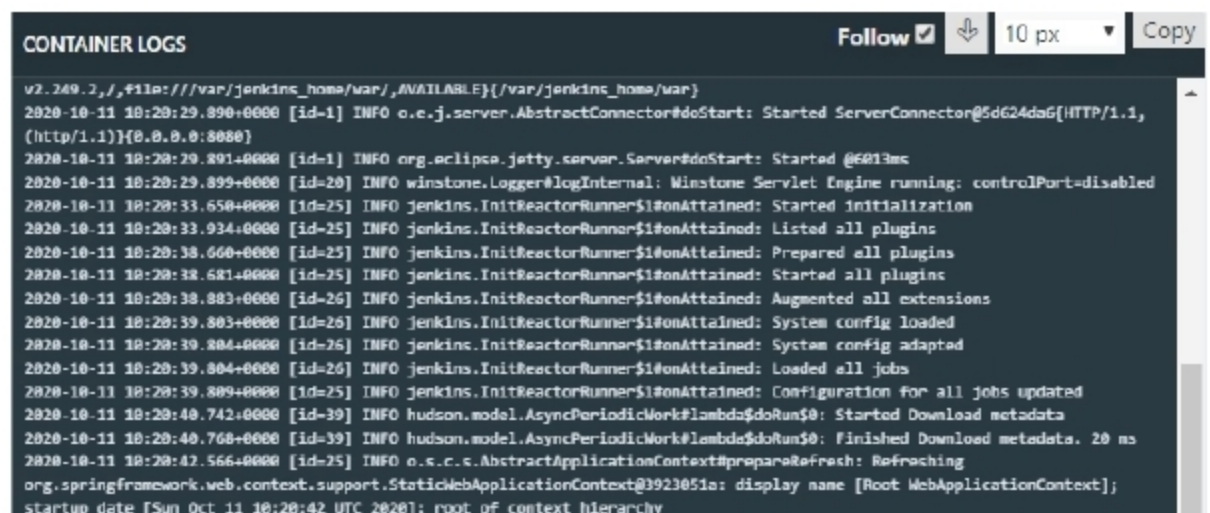
Shriya Kabra@LAPTOP-ELEUSLI8 MINGW64 /c:/Program Files/Docker Toolbox
$ ping www.google.com_

Pinging www.google.com [172.217.166.100] with 32 bytes of data:
Reply from 172.217.166.100: bytes=32 time=64ms TTL=117
Reply from 172.217.166.100: bytes=32 time=64ms TTL=117
Reply from 172.217.166.100: bytes=32 time=63ms TTL=117
Reply from 172.217.166.100: bytes=32 time=63ms TTL=117

Ping statistics for 172.217.166.100:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 63ms, Maximum = 64ms, Average = 63ms

```

Figure 4: Basic commands supported by Docker Quickstart Terminal



```

CONTAINER LOGS
Follow 10 px Copy

v2.249.2,,f11e:///var/jenkins_home/var/AVAILABLE/[/var/jenkins_home/var]
2020-10-11 10:20:29.890+0000 [id=1] INFO o.e.j.server.AbstractConnector@doStart: Started ServerConnector@5d624da6[HTTP/1.1,
(HTTP/1.1)]{0.0.0.0:8080}
2020-10-11 10:20:29.891+0000 [id=1] INFO org.eclipse.jetty.server.Server@doStart: Started @6013mc
2020-10-11 10:20:29.899+0000 [id=20] INFO winstone.Logger#logInternal: Winston Servlet Engine running: controlPort=disabled
2020-10-11 10:20:33.650+0000 [id=25] INFO jenkins.InitReactorRunner$1@onAttained: Started initialization
2020-10-11 10:20:33.934+0000 [id=25] INFO jenkins.InitReactorRunner$1@onAttained: Listed all plugins
2020-10-11 10:20:38.660+0000 [id=25] INFO jenkins.InitReactorRunner$1@onAttained: Prepared all plugins
2020-10-11 10:20:38.681+0000 [id=25] INFO jenkins.InitReactorRunner$1@onAttained: Started all plugins
2020-10-11 10:20:38.803+0000 [id=26] INFO jenkins.InitReactorRunner$1@onAttained: Augmented all extensions
2020-10-11 10:20:39.803+0000 [id=26] INFO jenkins.InitReactorRunner$1@onAttained: System config loaded
2020-10-11 10:20:39.804+0000 [id=26] INFO jenkins.InitReactorRunner$1@onAttained: System config adapted
2020-10-11 10:20:39.804+0000 [id=26] INFO jenkins.InitReactorRunner$1@onAttained: Loaded all jobs
2020-10-11 10:20:39.809+0000 [id=25] INFO jenkins.InitReactorRunner$1@onAttained: Configuration for all jobs updated
2020-10-11 10:20:40.742+0000 [id=39] INFO hudson.model.AsyncPeriodicWork$lambda$doRun$0: Started Download metadata
2020-10-11 10:20:40.768+0000 [id=39] INFO hudson.model.AsyncPeriodicWork$lambda$doRun$0: Finished Download metadata. 20 ms
2020-10-11 10:20:42.566+0000 [id=25] INFO o.s.c.s.c.AbstractApplicationContext#prepareRefresh: Refreshing
org.springframework.web.context.support.StaticWebApplicationContext@3923051a: display name [Root WebApplicationContext];
startup date [Sun Oct 11 10:20:42 UTC 2020]; root of context hierarchy

```

Figure 5: Logs of the Jenkins container

environment variables, exposed ports, networks, etc. We can always go to this panel and change the settings as required. Thus, with the Kitematic GUI, it has become easier to create and manage containers. Figure 6 shows the *Settings* of the Jenkins container.

Image migration

Migration of Docker images from Docker Toolbox to Docker Desktop can be easily done by saving the image as a tar file by following these steps:

1. Create a separate directory for Docker images and give it all the permissions as in the commands given below:

```
$mkdir ~/docker-images
$cd ~/docker-images
$chmod 777 ./
```

2. Use the *docker save* command for creating a tar file:

```
$docker save <image_id> -o ./<image_
name>.tar
```